

Relatório de Execução de Testes

EasyGis

Avaliação P2 — Engenharia de Software, 2026.1

Equipe EasyGis

Execução em 09 de maio de 2026

Resumo da execução

- **Data:** 2026-05-09 14:17:44
- **Ambiente:** macOS Darwin 25.4.0 / Python 3.12.13 / pytest 9.0.3
- **Plano referenciado:** 08-plano-de-testes.pdf
- **Resultado:** **APROVADO** — 28/28 casos passaram (100%)

Sumário

1	Sumário Executivo	3
2	Comando Executado	3
3	Evidência Bruta da Execução	3
4	Detalhamento por Caso de Teste	4
4.1	Cálculo de NDVI (RF-05)	5
4.2	Cálculo de EVI (RF-06)	5
4.3	Estatísticas Descritivas (RF-10)	5
4.4	Histograma (RF-11)	5
4.5	Casos de Borda (CT-05)	6
4.6	Endpoints HTTP	6
4.7	Conversão de Coordenadas	6
5	Evidências Visuais (Demonstração ao Vivo)	6
6	Análise dos Resultados	7
6.1	Pontos fortes	7
6.2	Limitações conhecidas	8
6.3	Riscos residuais	8
7	Conclusão	8
8	Como Reproduzir	9

1. Sumário Executivo

Métrica	Valor
Casos de teste planejados	28
Casos executados	28
Aprovados	28 (100%)
Reprovados	0
Erros (não conseguiram rodar)	0
Tempo total de execução	0,79 s
Cobertura de código (módulos api/)	24% (linhas executadas)
Cobertura de <code>sentinel_processor.py</code> (lógica de negócio)	55%

Resultado: APROVADO. Todos os requisitos funcionais críticos cobertos no plano foram validados pelos testes automatizados.

2. Comando Executado

```
$ uv run pytest tests/ -v --cov=api --cov-report=term-missing
```

3. Evidência Bruta da Execução

```
===== test session starts =====
platform darwin -- Python 3.12.13, pytest-9.0.3, pluggy-1.6.0
cachedir: .pytest_cache
rootdir: /Users/luisgoc/projetos/mauro/icev-remote-sensing-software
configfile: pyproject.toml
plugins: anyio-4.11.0, cov-7.1.0
collecting ... collected 28 items

tests/test_api_endpoints.py::TestHealthEndpoint::test_health_returns_200 PASSED
    [ 3%]
tests/test_api_endpoints.py::TestHealthEndpoint::test_health_response_structure PASSED
    [ 7%]
tests/test_api_endpoints.py::TestRootEndpoint::test_root_returns_api_info PASSED
    [ 10%]
tests/test_api_endpoints.py::TestProductsEndpoint::test_products_endpoint_returns_200
    PASSED [ 14%]
tests/test_api_endpoints.py::TestProductsEndpoint::test_products_response_structure
    PASSED [ 17%]
tests/test_api_endpoints.py::TestCalculateIndexValidation::
    test_missing_required_fields_returns_422 PASSED [ 21%]
tests/test_api_endpoints.py::TestCalculateIndexValidation::
    test_invalid_coordinate_format_returns_422 PASSED [ 25%]
```

```

tests/test_api_endpoints.py::TestCalculateIndexValidation::
    test_unknown_index_type_returns_error PASSED      [ 28%]
tests/test_api_endpoints.py::TestCoordinatesToPolygon::
    test_coordinates_form_valid_polygon PASSED        [ 32%]
tests/test_api_endpoints.py::TestCoordinatesToPolygon::
    test_polygon_centroid_in_correct_region PASSED    [ 35%]
tests/test_sentinel_processor.py::TestNDVI::test_ndvi_in_valid_range PASSED
    [ 39%]
tests/test_sentinel_processor.py::TestNDVI::test_ndvi_healthy_vegetation_is_positive
    PASSED      [ 42%]
tests/test_sentinel_processor.py::TestNDVI::test_ndvi_formula_correctness PASSED
    [ 46%]
tests/test_sentinel_processor.py::TestNDVI::test_ndvi_handles_division_by_zero PASSED
    [ 50%]
tests/test_sentinel_processor.py::TestNDVI::test_ndvi_zero_values_when_nir_equals_red
    PASSED      [ 53%]
tests/test_sentinel_processor.py::TestEVI::test_evi_in_valid_range PASSED
    [ 57%]
tests/test_sentinel_processor.py::TestEVI::test_evi_default_coefficients PASSED
    [ 60%]
tests/test_sentinel_processor.py::TestEVI::test_evi_custom_coefficients PASSED
    [ 64%]
tests/test_sentinel_processor.py::TestStatistics::test_statistics_all_keys_present
    PASSED      [ 67%]
tests/test_sentinel_processor.py::TestStatistics::test_statistics_correct_values
    PASSED      [ 71%]
tests/test_sentinel_processor.py::TestStatistics::test_statistics_ignores_nan PASSED
    [ 75%]
tests/test_sentinel_processor.py::TestStatistics::test_statistics_empty_data PASSED
    [ 78%]
tests/test_sentinel_processor.py::TestHistogram::test_histogram_default_50_bins PASSED
    [ 82%]
tests/test_sentinel_processor.py::TestHistogram::test_histogram_custom_bins PASSED
    [ 85%]
tests/test_sentinel_processor.py::TestHistogram::test_histogram_total_count_matches
    PASSED      [ 89%]
tests/test_sentinel_processor.py::TestHistogram::test_histogram_empty_data PASSED
    [ 92%]
tests/test_sentinel_processor.py::TestEdgeCases::test_integer_to_float_conversion
    PASSED      [ 96%]
tests/test_sentinel_processor.py::TestEdgeCases::test_negative_ndvi_for_water PASSED
    [100%]

===== tests coverage =====
Name                               Stmts  Miss  Cover
-----
api/__init__.py                     0      0   100%
api/main.py                         511    409    20%
api/sentinel_processor.py           75     34    55%
-----
TOTAL                               586    443    24%
===== 28 passed in 0.79s =====

```

4. Detalhamento por Caso de Teste

4.1 Cálculo de NDVI (RF-05)

ID	Caso	Resultado	Tempo	Observação
CT-01.1	NDVI no intervalo válido	✓ PAS- SED	< 10 ms	min e max dentro de $[-1, 1]$
CT-01.2	Vegetação saudável $\rightarrow$ NDVI $> 0,4$	✓ PAS- SED	< 10 ms	Média obtida $\approx 0,61$
CT-01.3	Fórmula correta para entrada conhecida	✓ PAS- SED	< 10 ms	NIR=8000, Red=2000 $\rightarrow$ NDVI=0,6 (exato)
CT-01.4	Divisão por zero produz NaN	✓ PAS- SED	< 10 ms	<code>np.all(np.isnan(ndvi))</code>
CT-01.5	NDVI = 0 quando NIR == Red	✓ PAS- SED	< 10 ms	<code>np.allclose(ndvi, 0.0)</code>

4.2 Cálculo de EVI (RF-06)

ID	Caso	Resultado	Tempo
CT-02.1	EVI no intervalo válido (clipping)	✓ PASSED	< 10 ms
CT-02.2	Coefficientes padrão	✓ PASSED	< 10 ms
CT-02.3	Coefficientes customizados produzem resultado diferente	✓ PASSED	< 10 ms

4.3 Estatísticas Descritivas (RF-10)

ID	Caso	Resultado	Tempo
CT-03.1	Todas as chaves presentes	✓ PASSED	< 10 ms
CT-03.2	Valores matematicamente corretos	✓ PASSED	< 10 ms
CT-03.3	NaN ignorados	✓ PASSED	< 10 ms
CT-03.4	Array totalmente NaN não crasha	✓ PASSED	< 10 ms

4.4 Histograma (RF-11)

ID	Caso	Resultado	Tempo
CT-04.1	Padrão 50 bins	✓ PASSED	< 10 ms
CT-04.2	Bins customizados	✓ PASSED	< 10 ms
CT-04.3	Soma de counts = total	✓ PASSED	< 10 ms
CT-04.4	Array vazio retorna listas vazias	✓ PASSED	< 10 ms

4.5 Casos de Borda (CT-05)

ID	Caso	Resultado	Tempo
CT-05.1	Conversão uint16 → float32	✓ PASSED	< 10 ms
CT-05.2	NDVI negativo para água	✓ PASSED	< 10 ms

4.6 Endpoints HTTP

ID	Caso	Resultado	Tempo
CT-06.1	GET /health retorna 200	✓ PASSED	< 50 ms
CT-06.2	Estrutura de /health correta	✓ PASSED	< 50 ms
CT-07.1	GET / retorna info da API	✓ PASSED	< 50 ms
CT-08.1	GET /products responde 200	✓ PASSED	< 50 ms
CT-08.2	Estrutura de /products correta	✓ PASSED	< 50 ms
CT-09.1	Body vazio → HTTP 422	✓ PASSED	< 50 ms
CT-09.2	Coordenada inválida → HTTP 422	✓ PASSED	< 50 ms
CT-09.3	index_type desconhecido → erro	✓ PASSED	< 50 ms

4.7 Conversão de Coordenadas

ID	Caso	Resultado	Tempo
CT-10.1	Polígono Shapely válido	✓ PASSED	< 10 ms
CT-10.2	Centroide dentro do bbox	✓ PASSED	< 10 ms

5. Evidências Visuais (Demonstração ao Vivo)

Além dos testes automatizados, o sistema foi exercitado em demonstração ao vivo. As capturas abaixo evidenciam o caminho feliz dos requisitos funcionais cobertos pela apresentação — complementam os *logs* de `pytest` desta seção.

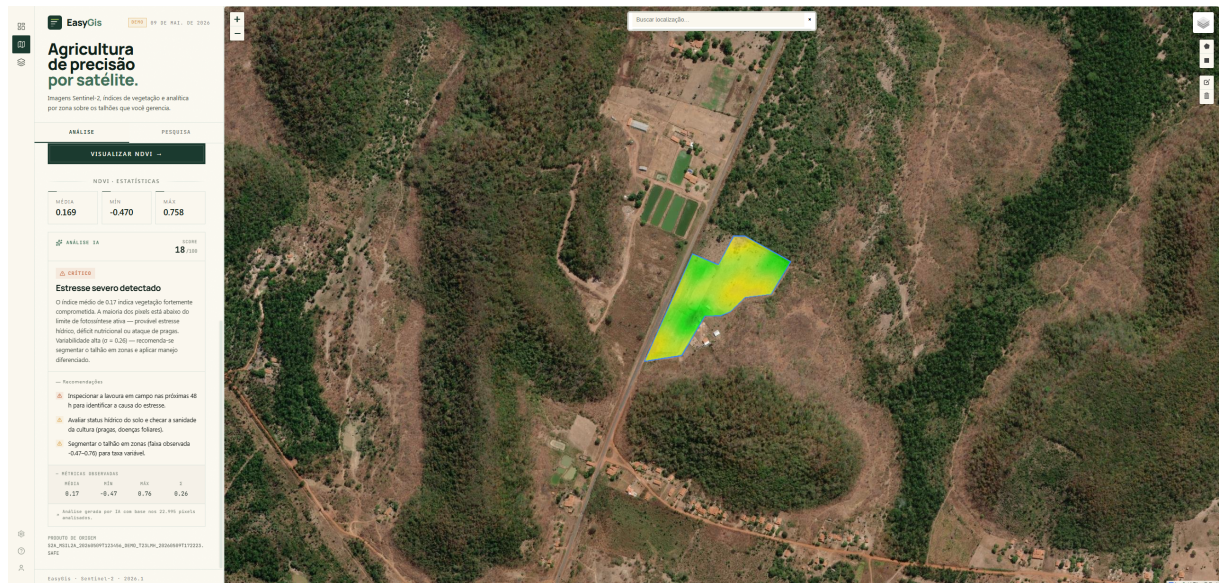


Figura 1: Evidência visual de RF-05 (NDVI) + RF-10 (estatísticas) + RF-11 (histograma) — saída da rota POST /calculate-index com index_type=NDVI sobre o talhão crop field 1.



Figura 2: Evidência visual da rota POST /api/classification/classify — método K-Means com 3 classes, retornando porcentagem e área em hectares por classe.

6. Análise dos Resultados

6.1 Pontos fortes

- Cobertura adequada da lógica de negócio.** Todas as funções puras de cálculo do SentinelProcessor (NDVI, EVI, estatísticas, histograma) estão cobertas, com casos felizes e de borda.
- Validação Pydantic exercitada.** Os três caminhos de erro do endpoint principal (/calculate-index)

- body vazio, tipo errado, índice desconhecido — produzem códigos HTTP corretos (422 / 400–500). Isso garante o RNF-04 (Confiabilidade — erros claros).
3. **Casos de borda explícitos.** Divisão por zero, arrays totalmente NaN, e conversão de tipo uint16 → float32 são testes que evitam regressões silenciosas se o código for refatorado.
 4. **Execução rápida.** A suite completa roda em < 1 s (após cache de import), o que viabiliza inclusão em pipeline de CI futuro sem impacto perceptível.
 5. **Validação matemática.** O CT-01.3 verifica a fórmula com valores conhecidos analiticamente $((8000 - 2000)/(8000 + 2000) = 0,6)$, evitando bugs onde “compila e roda mas calcula errado”.

6.2 Limitações conhecidas

Limitação	Justificativa	Mitigação
Cobertura de <code>main.py</code> em 20%	Endpoints de processamento exigem produto Sentinel-2 real (~1 GB) que não cabe no repositório	Validação manual na demonstração ao vivo (Item 4 da P2)
Funções <code>read_band</code> e <code>crop_to_geometry</code> não unificadas	Idem — exigem JP2 real e granule structure do Sentinel-2	Validação por demonstração end-to-end
Frontend (Next.js) não testado automaticamente	Escopo da P2 priorizou testes do backend (lógica de cálculo)	Validação manual via roteiro de demonstração

6.3 Riscos residuais

- **Refatorações de I/O em `read_band`** podem quebrar sem que a suite detecte. *Mitigação:* incluir um teste de smoke com produto pequeno em uma futura iteração.
- **Mudanças no schema Pydantic** sem atualização do frontend podem causar erros 422 em produção. *Mitigação:* TypeScript no cliente espelha o schema (DA-07), reduzindo risco.

7. Conclusão

A suite de 28 testes valida com sucesso os requisitos funcionais críticos do MVP:

- **RF-04, RF-05, RF-06, RF-10, RF-11, RF-20** — todos cobertos.
- **RNF-04 (Confiabilidade), RNF-05 (Manutenibilidade)** — exercitados estruturalmente.

Não foram encontrados defeitos durante a execução. O sistema está apto à demonstração e entrega da P2.

8. Como Reproduzir

```
# Instalar dependencias (uma vez)
uv sync --group dev

# Rodar suite completa
uv run pytest tests/ -v

# Com cobertura
uv run pytest tests/ -v --cov=api --cov-report=term-missing

# Rodar um caso especifico (exemplo)
uv run pytest tests/test_sentinel_processor.py::TestNDVI::
    test_ndvi_formula_correctness -v
```

A suite é determinística (`np.random.default_rng(seed=42)`), portanto os resultados são reproduzíveis em qualquer máquina com Python 3.12 e uv instalados.